SHR Converter 0.0.1

Signal Hound Spike `.shr` recordings to CSV and native Veusz documents
User and technical manual

William W. Wallace
NRAO/GBO
wwallace@nrao.edu

2026-10-28

Program	Version
`shr_converter.py` (launcher)	0.0.1
`shr_converter_gui.py`	0.0.1
`shr_converter_core.py`	0.0.1
`vszh5_writer.py`	0.0.20
py-shr-parser (reader)	1.0.0

Contents

1 Overview **2**

2 Installation **2**

3 The window **2**
3.1 Menus and keys . . . 3
3.2 Workflow . . . 3

4 Outputs **4**
4.1 CSV . . . 4
4.2 Veusz document . . . 4

5 Statistics and their derivations **4**
5.1 Frequency of a bin . . . 4
5.2 Power mean, dB mean and the 2.51 dB bias . . . 4
5.3 mV recordings . . . 6
5.4 Standard deviation from running sums . . . 6
5.5 Exact merging of blocks . . . 6
5.6 Spectrogram pixels . . . 6
5.7 Lossless numbers . . . 6

6 Time axes and the even grid **7**

7 Long captures **7**

8 Performance, parallelism, GPU and memory **9**

9 Robustness **9**

10 Command line **9**

11 vszh5_writer 0.0.20 **10**

12 Verification **10**

13 Data gaps **11**

A Options (ConvertOptions) and their defaults **11**

B File-header fields **12**

C Modules and functions used, and why **12**

References **13**

1 Overview

Spike, Signal Hound's analyzer software, records sweeps to `.shr` files: a 472-byte file header, then per sweep a 48-byte sweep header (time stamp, GPS latitude/longitude/altitude, ADC-overflow flag) and the amplitudes as 32-bit floats [1],[2]. The SHR Converter reads these files with py-shr-parser [1] and writes CSV tables and a native Veusz document (`.vszh5`) through the author's `vszh5_writer` [13], optionally rendered by Veusz [10] to a bookmarked PDF and PNGs. Everything is streamed in blocks, so memory is set by a RAM budget and not by the file, and the blocks run in parallel worker processes, with the numerical part on the GPU when CuPy is present.

2 Installation

1. `mamba env create -f environment.yml` — Python 3.12; pip installs `requirements.txt` (py-shr-parser is on PyPI only).
2. `mamba activate shr-converter`
3. Optional: `mamba install -c conda-forge veusz` (or the Veusz installer) to open and render; `mamba install -c conda-forge cupy` for an NVIDIA GPU.
4. `python shr_converter.py --selftest --gui-smoke` — 38 engine checks and 11 GUI checks.
5. `python shr_converter.py` starts the GUI. On Windows: `create_environment.bat` once, then `run_shr_converter.bat` (files may be dropped on it) and `run_selftest.bat`.

The launcher imports nothing heavy at its top level: worker processes started with *spawn* re-run the main script's top level, and this keeps Qt (about 50 MB per process) out of them.

3 The window

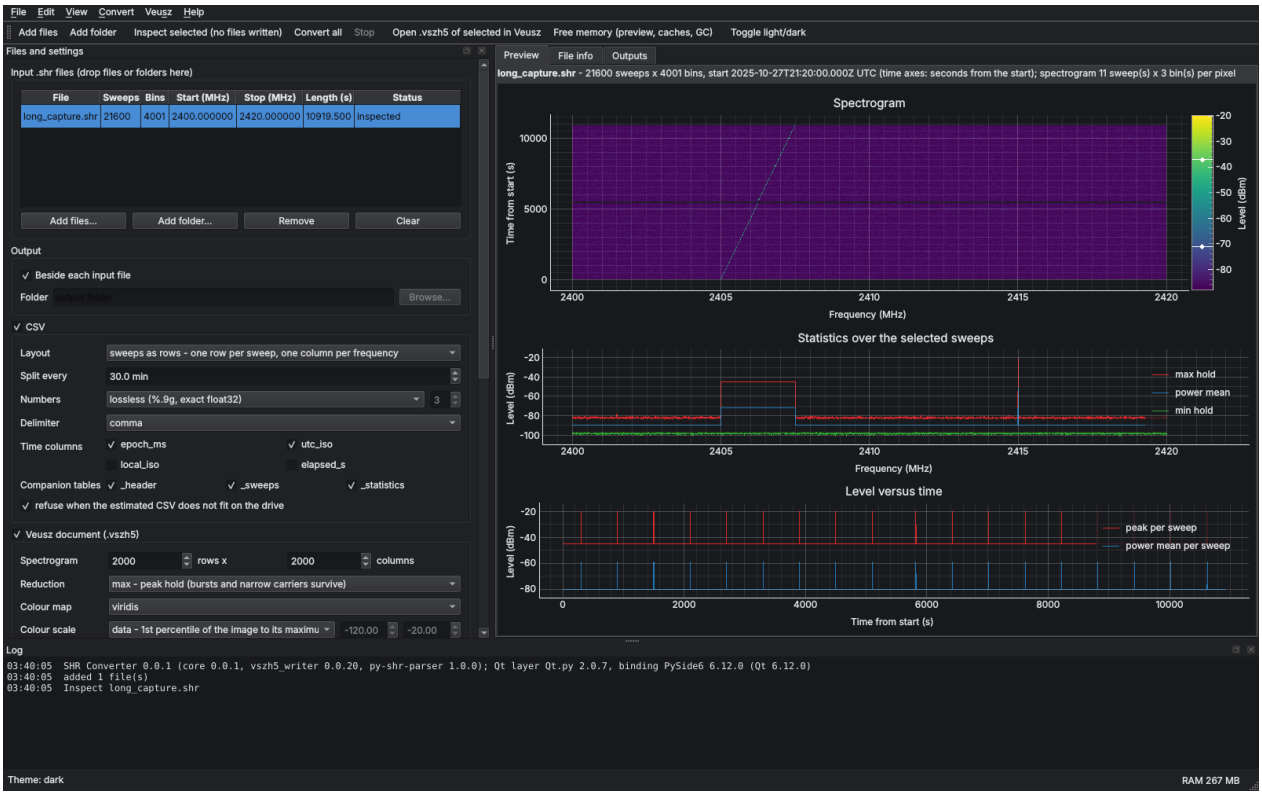


Figure 1: Dark theme, a synthetic 3-hour capture inspected: input list and settings (left), preview (centre), log (bottom).

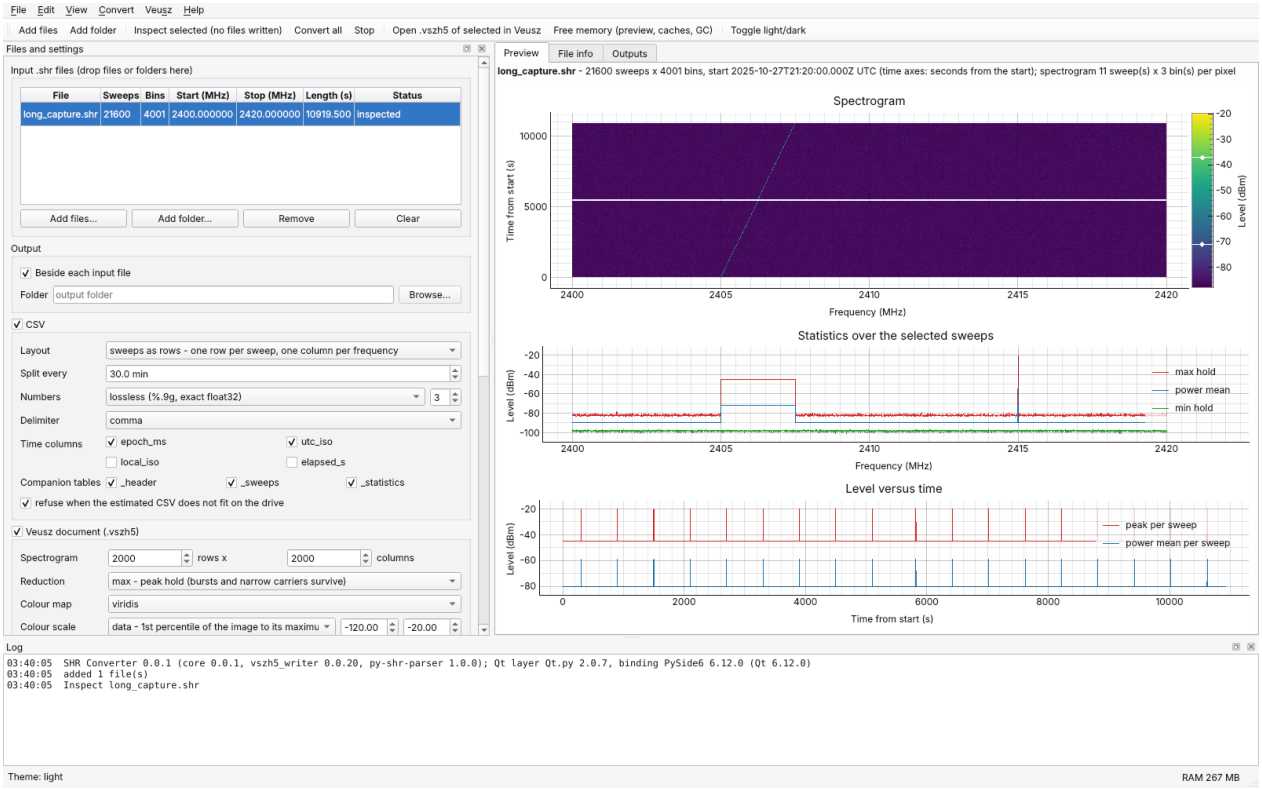


Figure 2: Light theme (View > Theme, or Ctrl+T to toggle).

3.1 Menus and keys

File Add files (Ctrl+O), Add folder (Ctrl+Shift+O), Recent inputs, Remove (Del), Clear, Output folder, Open output folder (Ctrl+Shift+D), Save/Load settings (JSON), Save log, Quit.

Edit Copy file info, copy/clear log, **Free memory** (Ctrl+Shift+M).

View **Theme: System / Light / Dark**, toggle light/dark (Ctrl+T), auto-range (Ctrl+0), panels, toolbar, full screen (F11).

Convert Inspect (Ctrl+I), Convert all (F5), Convert selected (Shift+F5), Stop (Esc), Show plan (Ctrl+P), Self-test.

Veusz Open the `.vszh5` in Veusz (Ctrl+Shift+Z), Render to PDF/PNG (Ctrl+R), Open PDF.

Help Manual (F1), keyboard shortcuts, About (author, licence, every loaded version), About Qt.

3.2 Workflow

1. Add files. Each header is read at once (sweeps, bins, span, length); notes report cut-short recordings.
2. **Inspect** runs the complete statistics pass without writing any file — the same code as Convert — and shows the preview. The *File info* tab shows the plan (Figure 3).
3. **Convert**: one worker pool serves the whole batch; each file's preview arrives when it is done. **Stop** ends after the current block and deletes that file's partial outputs.

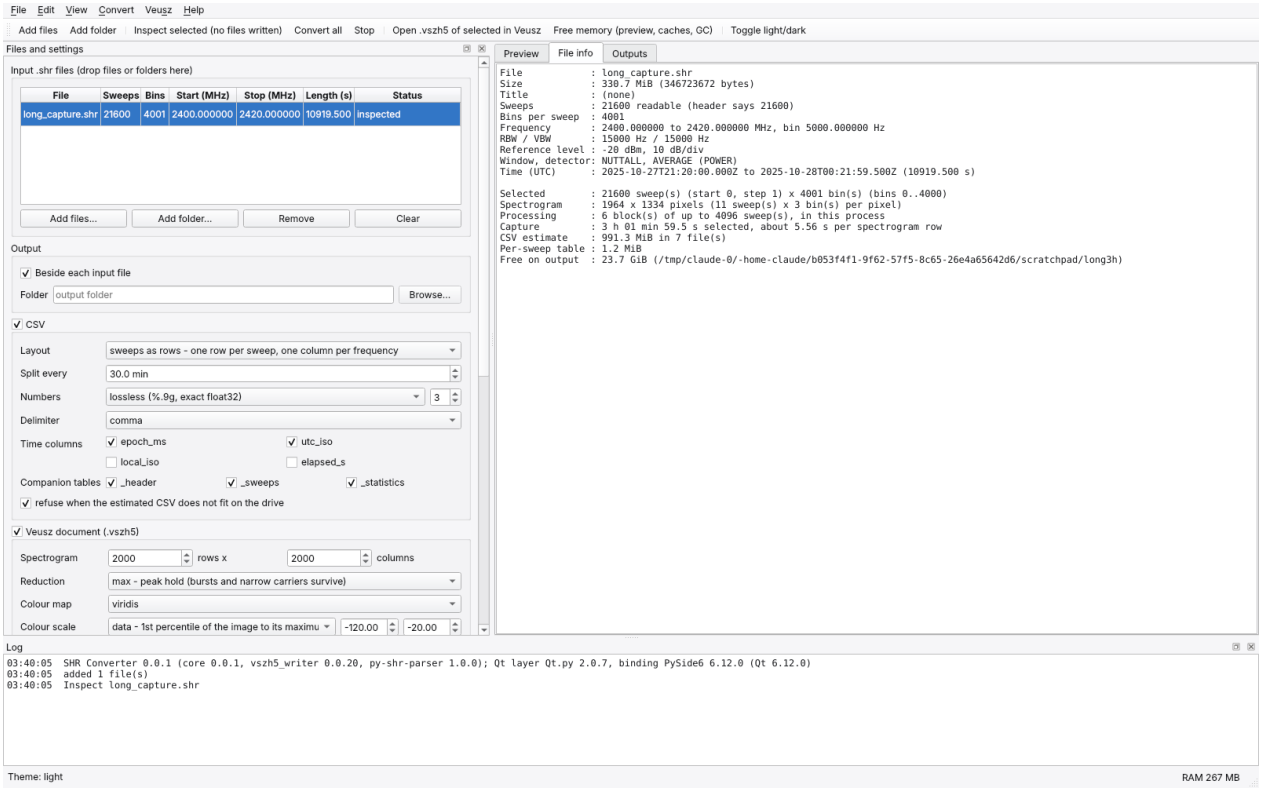


Figure 3: File info: header, plan, CSV estimate, seconds per spectrogram row and free disk space.

4 Outputs

4.1 CSV

`sweeps_as_rows` one row per sweep: index, time columns, GPS, overflow, then one column per frequency (header: the bin frequencies in Hz). The default.

`frequencies_as_rows`
one row per bin: frequency, then one column per sweep. Needs every selected sweep of a column block at once, so the file is read once per block (the plan says how often).

`long` one row per sample: index, time columns, frequency, amplitude.

Companion tables: `_header.csv` (every header field with its unit, derived values, CSV parts with their time spans, versions), `_sweeps.csv` (per sweep: time, GPS, overflow, peak, peak frequency, power mean) and `_statistics.csv` (per bin: max hold, min hold, power mean, dB mean, standard deviation, count). Files are written to private `.part` names and published only when complete; a CSV that another program holds open (Excel on Windows) is kept under a time-stamped name. Excel's limits (16 384 columns, 1 048 576 rows [6]) are reported by the plan.

4.2 Veusz document

Page	Title (PDF bookmark)
`spectrogram`	Spectrogram
`statistics`	Spectrum statistics
`selected_sweeps`	Selected sweeps
`level_vs_time`	Level versus time
`gps_track`	GPS track
`file_header`	File header

5 Statistics and their derivations

Let $x_k(f)$ be the level of bin f in sweep k, $k = 1 \ldots N$.

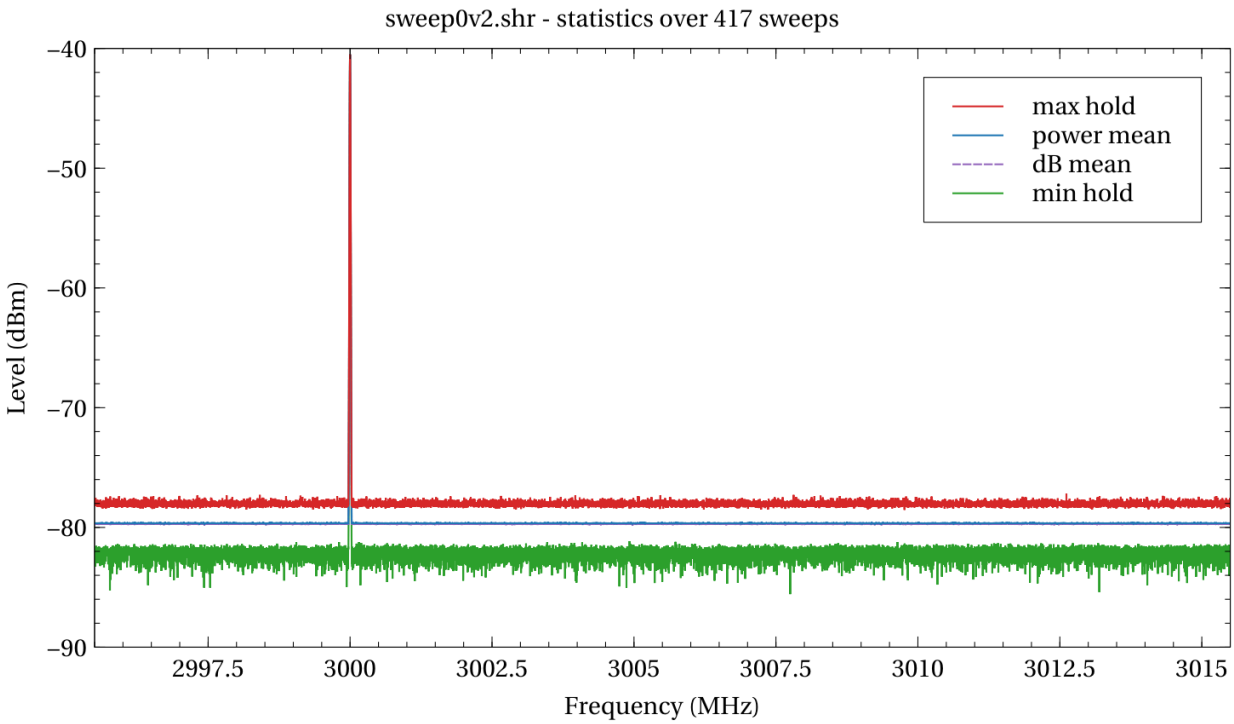


Figure 4: Statistics page of Spike's own test file sweep0v2.shr (417 sweeps): max hold, power mean, dB mean, min hold.

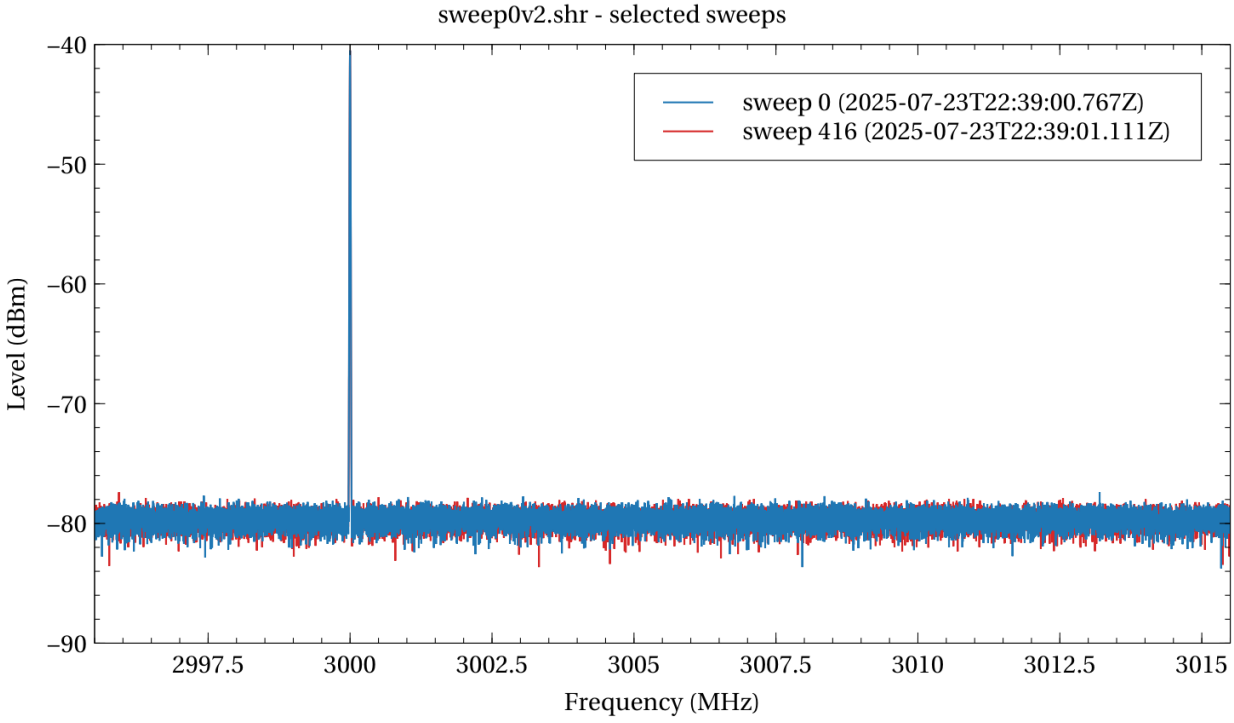


Figure 5: Selected-sweeps page: first and last sweep (the peak sweep is the first here).

For noise the dB mean reads low [4]. With a Rayleigh envelope the power p is exponential with mean μ, $\rho(p) = \mu^{-1} e^{-p/\mu}$, and with $u = p/\mu$

$$\mathrm{E}[\ln p] = \ln \mu + \int_0^\infty \ln u \, e^{-u} \, \mathrm{d}u = \ln \mu + \Gamma'(1) = \ln \mu - \gamma, \tag{4}$$

so

$$\mathrm{E}[10 \log_{10} p] - 10 \log_{10} \mu = -\frac{10\,\gamma}{\ln 10} = -2.5068 \text{ dB}, \qquad \gamma = 0.5772156649\ldots \tag{5}$$

(a 10^7-sample simulation gave -2.5067 dB). The converter therefore reports both: the power mean is the physical mean power, the dB mean matches Spike's log averaging.

5.3 mV recordings

$$V_{\mathrm{rms}}(f) = \sqrt{\frac{1}{N} \sum_k x_k(f)^2} \quad [\mathrm{mV}]. \tag{6}$$

5.4 Standard deviation from running sums

$$s^2 = \frac{S_2 - S_1^2/N}{N-1}, \qquad S_1 = \sum_k x_k, \; S_2 = \sum_k x_k^2. \tag{7}$$

In float64 the cancellation error is about $\varepsilon \, \bar{x}^2 / s^2 \approx 10^{-16} \times 80^2 / 2^2 \approx 2 \times 10^{-13}$ (relative), negligible for dBm levels.

5.5 Exact merging of blocks

Each block returns max, min, S_1, S_2, $\sum 10^{x/10}$ and N per bin; the file's values follow exactly: maxima of maxima, minima of minima, sums of sums, e.g. $\bar{P}_{A \cup B} = 10 \log_{10} \big((\Sigma_A + \Sigma_B) / (N_A + N_B) \big)$. Parallel and serial conversions give identical results (self-test).

5.6 Spectrogram pixels

A pixel covers n_r sweeps $\times$ n_c bins. Peak hold is the block maximum; the power mean is

$$\bar{P}_{ij} = 10 \log_{10} \Big(\frac{1}{n_r n_c} \sum_{k \in r_i} \sum_{f \in c_j} 10^{x_k(f)/10} \Big) = 10 \log_{10} \Big(\frac{1}{n_r} \sum_{k \in r_i} \Big[\frac{1}{n_c} \sum_{f \in c_j} 10^{x_k(f)/10} \Big] \Big), \tag{8}$$

computed as the column mean per sweep, then summed over the sweeps of the row (exact, since every sweep of a row has the same n_c). A row may span two worker blocks; its partial sums are added by the main process.

5.7 Lossless numbers

Binary32 has $p = 24$ significand bits; a decimal string round-trips with [5],[12]

$$d = \lceil p \log_{10} 2 \rceil + 1 = \lceil 7.22 \rceil + 1 = 9 \tag{9}$$

significant digits, hence `%.9g`. Every value of every layout was compared with py-shr-parser bit for bit.

6 Time axes and the even grid

Time stamps are milliseconds since 1970 UTC from the recording PC's clock [2]. The `.vszh5` time axis is UTC or local date/time, elapsed seconds or sweep index (`auto`: date/time for captures of 60 s or more). Spectrogram rows are put on an even grid by `vszh5_writer.regularize_rows`: the step is fitted over the whole span, $\Delta = (c_{\text{last}} - c_0)/s_{\text{last}}$ with slots $s_k = \text{round}((c_k - c_0)/\Delta)$; rows that already lie within $\Delta/4$ of the grid are kept; pauses become blank (NaN) rows; crowded time stamps are spread evenly. Veusz draws images with uneven axes through a non-linear path: on a 3-hour spectrogram 30.3 s and 3.2 GB, against 2.7 s and 0.31 GB on the even grid.

7 Long captures

- **CSV split by time**: `_part001.csv`, …, one per N minutes of recording, each with the header line; slices follow the running maximum of the time stamps, so a clock step back never sends a sweep to an earlier file.
- **Disk check**: the CSV size is estimated (lossless: ≈ 12 bytes per value) and the conversion refuses to start when the drive cannot hold it.
- **Per-sweep table**: 57 bytes per sweep, memory-mapped from a private temporary folder when larger than a quarter of the RAM budget.
- **Thinned series**: level versus time and GPS in the `.vszh5` are reduced to `time_series_max_points` (100 000) — peak hold for the peak, power mean for the band level, the frequency of the strongest sweep, any overflow — so a single burst is never averaged away.
- **Time per row**: 2000 rows over 3 h is 5.4 s per row; the plan states it.

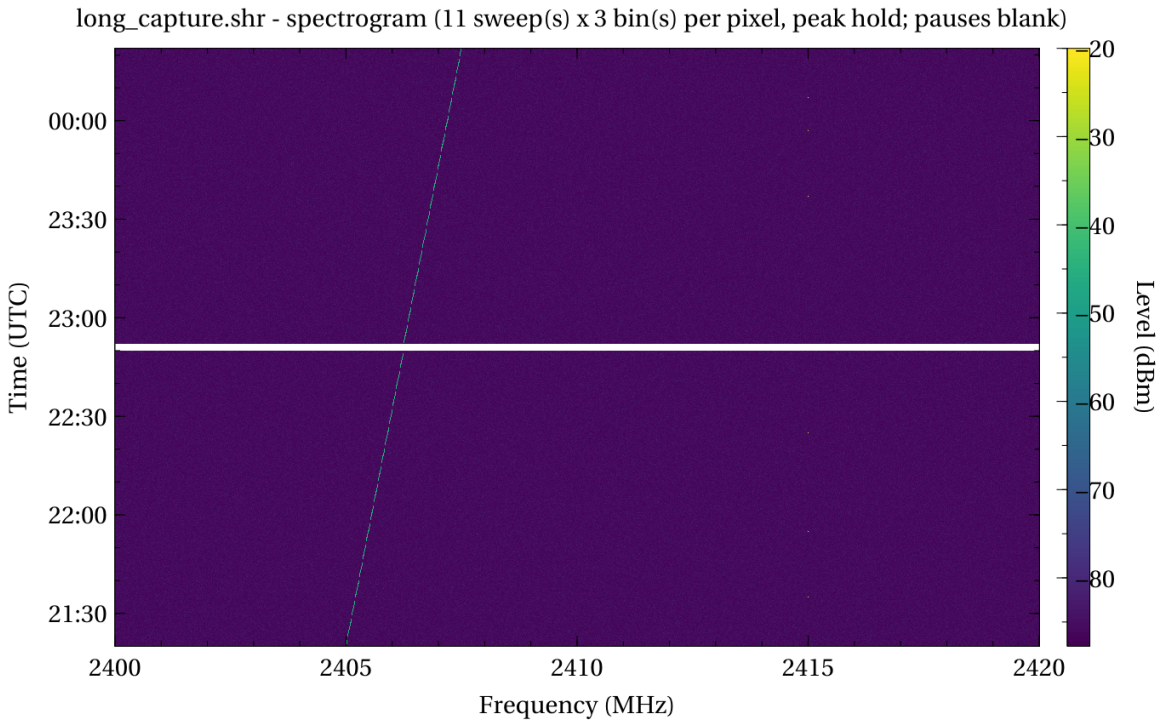


Figure 6: Synthetic 3-hour capture: drifting carrier, a 2 ms burst every 10 min at 2415 MHz, a 2-minute pause (blank band).

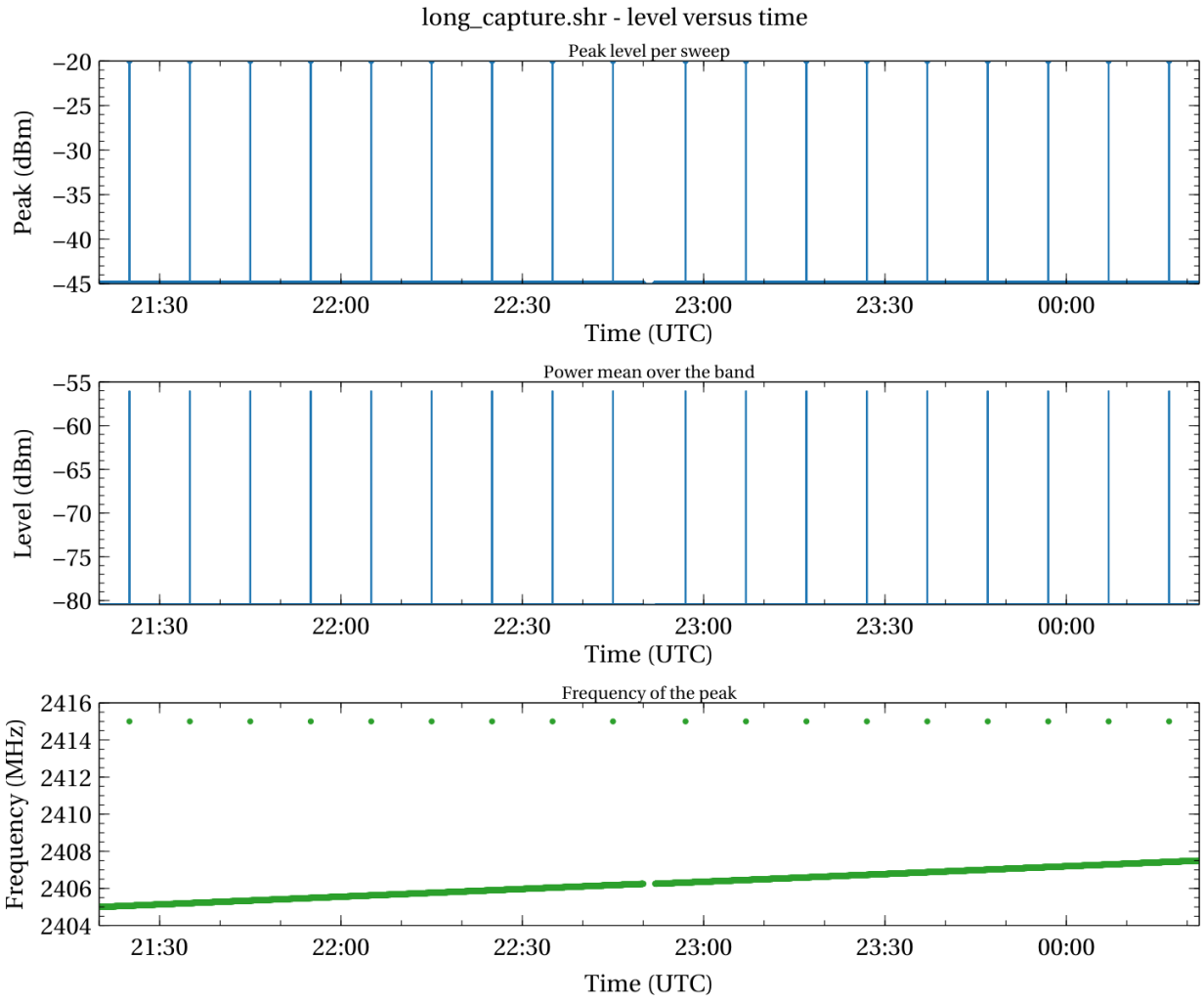


Figure 7: Level versus time of the same capture: every burst present, the pause visible.

8 Performance, parallelism, GPU and memory

- Blocks of sweeps are read, formatted and reduced in worker processes (*spawn*, never fork a GUI), at most 2×workers blocks in flight, merged in order [8]. Default workers: CPUs −1 (at most 61 on Windows).
- Block size: $B = \lfloor (M - M_{\text{stats}})/(s\, n_c\, \beta) \rfloor$ with budget M, $s = 2w + 1$ slots, n_c bins and β bytes per sample in flight (30 for `sweeps_as_rows`); the statistics work in sub-blocks of 2 M samples, so their float64 temporaries do not grow with the block.
- GPU: CuPy [9] runs the statistics and spectrogram reduction per sub-block; any GPU error repeats the block on the CPU (tested with a stand-in). Text formatting stays on the CPU.
- Edit > Free memory: garbage collection, CuPy pools, `malloc_trim` / `EmptyWorkingSet`; previews are pickled to disk, one in memory.

Table 2: Measured with tests/long_capture_check.py and tests/memory_check.py (docs/validation).

Case (2 CPU cores, no GPU)	Data	Time	Peak memory
3 h, 2 sweeps/s x 4001 bins, 2 workers, with the Veusz PDF/PNG render	331 MiB to 981 MiB CSV, 7 files	25.8 s	879 MiB
3 h, 50 sweeps/s x 512 bins (540 000 sweeps), 2 workers	1.1 GiB to CSV (2 decimals)	52.8 s	526 MiB
3000 x 65 536 bins, RAM budget 256 MB, 2 workers	750 MiB to 2.2 GiB CSV	41.4 s	418 MiB

9 Robustness

- py-shr-parser raises `ShrFileParserWarning` when the size does not match the header and leaves the file open; the converter reads up to the last complete sweep (header count 0 is replaced by the sweeps present) and says so [2].
- Every output is written to a private `.part` and published atomically; Stop, an error or a refused disk check leaves no partial file.
- Published files get the usual permissions (writer 0.0.20 fix of the 0600 mode).

10 Command line

```
python shr_converter.py --cli [options] files-or-folders
```

Option	Meaning
`inputs`	.shr files or folders (folders: every *.shr inside)
`-o, --output-dir`	output folder (default: beside each input)
`--no-csv`	do not write the CSV files
`--layout`	CSV layout: sweeps_as_rows (default), frequencies_as_rows or long
`--fixed`	fixed decimals instead of lossless %.9g
`--delimiter`	CSV delimiter: comma (default), semicolon or tab
`--time-columns`	comma list of epoch_ms, utc_iso, local_iso, elapsed_s
`--no-companions`	skip _header/_sweeps/_statistics.csv
`--split-minutes`	new CSV file every N minutes of recording (0 = one file)
`--no-disk-check`	do not refuse when the CSV may not fit
`--no-vszh5`	do not write the Veusz document
`--image-size`	maximum spectrogram ROWSxCOLUMNS

Option	Meaning
`--reduction`	spectrogram pixels: max (peak hold, default), power_mean or min
`--colormap`	Veusz colour map, e.g. viridis, magma, heat
`--z`	data | reference | veusz | MIN,MAX
`--time-axis`	auto (date/time from 60 s on), utc, local, elapsed or index
`--sweeps-to-plot`	selected-sweeps page: first,last,peak and/or sweep indices
`--render`	render PDF (bookmarked) and PNG with Veusz
`--png`	PNG pages when rendering: first (default), all or none
`--veusz`	Veusz program (else VEUSZ_EXE, PATH, usual folders)
`--sweeps`	sweep selection START:STOP:STEP (STOP exclusive)
`--freq`	frequency range FMIN:FMAX in MHz
`-j, --workers`	worker processes (0 = CPUs - 1)
`--ram-mb`	RAM budget for blocks
`--compute`	statistics on auto (CuPy GPU when present), cpu or gpu (falls back to CPU)
`--series-points`	maximum points of the level-vs-time and GPS series in the .vszh5
`--sweep-table`	per-sweep table in RAM or memory-mapped on disk (auto: by size)
`--info`	print header, plan and notes only
`--selftest`	run the self-test and exit
`--real-file`	also check this real .shr in the self-test
`--version`	show program's version number and exit

11 vszh5_writer 0.0.20

Two releases were both numbered 0.0.18: the R&S FSP logger 0.9.0 (render fixes) and the AB Power Meter Monitor 1.6.0 (`VeuszAppender`); 0.0.19 grew from the latter. 0.0.20 joins them and adds the 0600 file-mode fix, `png_background`, date-axis images, `block_reduce` and `regularize_rows`. Every public signature of 0.0.5, 0.0.10, both 0.0.18 and 0.0.19 is unchanged (checked by machine).

Program	Own suite with 0.0.20	Note
R&S FSP logger 0.9.0	44/44 (PyQt5, PySide6; Veusz 3.6.2, 4.2.1)	update the About test's expected version 0.0.18 -> 0.0.20
AB Power Meter Monitor 1.7.0	45/45, real renders	none
SA Time Stability Tools 0.0.3	7/7 and 22/22 in Veusz 4.2.1	none
QuickSyn package 0.0.10	44 pass, 3 pinned checks fail	same with 0.0.19: version string and pre-0.0.15 temporary name

12 Verification

`--selftest`: 38 checks (every CSV layout bit-exact with 1 and 2 workers; statistics, spectrogram and sub-block merging against direct NumPy; cut-short files; mV; GPU stand-in and fallback; split, disk check, disk-mapped table, thinning; locked file; cancel). `--gui-smoke`: 11 offscreen GUI checks including real worker processes. `pytest tests`: 28 tests with 30 sub-tests, real renders in Veusz 3.6.2 and 4.2.1 and Spike's own `sweep0v2.shr`. Logs: `docs/validation`.

13 Data gaps

Item	Status	Confidence
Real Spike files	sweep0v2.shr (v2, dBm) read bit-exactly	High
mV recordings	synthetic only	Medium
Title encoding	UTF-16LE if it looks like it, else UTF-8; no titled file seen	Medium
Channelized recordings	read as stored, not interpreted	Unknown
GPU	stand-in only; no CUDA device here	Medium
Windows	designed for it, tested on Linux	Medium

A Options (ConvertOptions) and their defaults

Field	Default
output_dir	""
write_csv	True
csv_layout	"sweeps_as_rows"
csv_number_format	"lossless"
csv_decimals	3
csv_delimiter	"comma"
csv_time_columns	('epoch_ms', 'utc_iso')
csv_header_table	True
csv_sweeps_table	True
csv_statistics_table	True
csv_split_minutes	0.0
check_disk_space	True
write_vszh5	True
vszh5_compress	True
image_max_rows	2000
image_max_columns	2000
image_reduction	"max"
colormap	"viridis"
z_mode	"data"
z_min	-120.0
z_max	-20.0
time_axis	"auto"
selected_sweeps	"first,last,peak"
stats_max_points	200000
time_series_max_points	100000
page_spectrogram	True
page_statistics	True
page_selected_sweeps	True
page_level_vs_time	True
page_gps	True
page_header	True
render	False
render_png	"first"
render_dpi	150
veusz_exe	""

Field	Default
sweep_start	0
sweep_stop	0
sweep_step	1
freq_min_hz	0.0
freq_max_hz	0.0
workers	0
ram_budget_mb	1024
compute	"auto"
sweep_table_storage	"auto"
flush_after_file	True

B File-header fields

Field	Unit	Meaning
signature		file signature (0xAA10)
version		SHR format version
data_offset	byte	offset of the first sweep
sweep_count		sweeps according to the header
sweep_length	bin	bins per sweep
first_bin_freq_hz	Hz	frequency of bin 0
bin_size_hz	Hz	bin spacing
center_freq_hz	Hz	centre frequency
span_hz	Hz	span
rbw_hz	Hz	resolution bandwidth
vbw_hz	Hz	video bandwidth
ref_level	dBm or mV	reference level
ref_scale		reference scale (DBM or MV)
div	dB	display division
window		RBW window
attenuation	dB	attenuation setting (index as stored by Spike)
gain		gain setting (index as stored by Spike)
detector		video detector
processing_units		video processing units
window_bandwidth	bin	window bandwidth in bins
decimation_type		time averaging type
decimation_detector		time averaging detector
decimation_count		sweeps (or seconds) per output trace
decimation_time_ms	ms	time averaging interval
channelize_enable		channelizer enabled
channel_output_units		channelizer units
channel_center_hz	Hz	channelizer centre
channel_width_hz	Hz	channelizer width

C Modules and functions used, and why

Module	Used	Why
`shr_parser.ShrFileParser`	open(), header, get_sweep_n()	reads every header and sweep; one instance per worker process [1]
`numpy`	float32/float64 arrays, ufunc.reduceat, memmap, percentile	statistics, block reductions, per-sweep table on disk
`cupy (optional)`	same array calls on the GPU	statistics and spectrogram reduction per sub-block [9]
`concurrent.futures`	ProcessPoolExecutor (spawn, max_tasks_per_child)	parallel blocks, bounded and in order [8]
`h5py via vszh5_writer`	VeuszWriter, image_page, xy_page, grid_page, text_page	native .vszh5 without Veusz
`vszh5_writer (0.0.20)`	block_reduce, regularize_rows, render_pages	thinning, even spectrogram grid, PDF/PNG with bookmarks
`Qt.py + PySide6`	QtCore, QtGui, QtWidgets	the GUI through the abstraction layer [11]
`pyqtgraph`	ImageItem, ColorBarItem, PlotDataItem	preview plots
`psutil`	Process().memory_info()	memory read-out and tests
`pypdf (via the writer)`	PdfWriter.add_outline_item	PDF bookmark tree
`ctypes`	malloc_trim (Linux), EmptyWorkingSet (Windows)	return freed memory to the OS

References

References

[1] T. Schmitz, “py-shr-parser: Python library for parsing Signal Hound SHR files, v1.0.0, WiSE Lab, Carnegie Mellon Univ., 2025. https://github.com/WiseLabCMU/py-shr-parser

[2] Signal Hound, “.shr file format specification,” BB Series Discussions forum, May–Aug. 2018. https://signalhound.com/?p=158701

[3] Signal Hound, “Open recording .shr file in MATLAB,” BB Series Discussions forum, Jul. 2019. https://signalhound.com/support/forums/topic/open-recording-shr-file-in-matlab/

[4] Hewlett-Packard, “Measuring Noise and Noise-like Digital Communications Signals with a Spectrum Analyzer,” Application Note 1303, lit. no. 5966-4008E. https://www.hpmemoryproject.org/an/pdf/an_1303.pdf

[5] IEEE, “IEEE Standard for Floating-Point Arithmetic,” IEEE Std 754-2019, pp. 1–84, Jul. 2019, doi: 10.1109/IEEESTD.2019.8766229. https://doi.org/10.1109/IEEESTD.2019.8766229

[6] Microsoft, “Excel specifications and limits.” https://support.microsoft.com/en-us/office/1672b34d-7043-467e-8e27-269d656771c3

[7] Keysight Technologies, “Spectrum Analysis Basics,” Application Note 150, 5952-0292. https://www.keysight.com/us/en/assets/7018-06714/application-notes/5952-0292.pdf

[8] Python Software Foundation, “concurrent.futures — Launching parallel tasks,” Python 3.12. https://docs.python.org/3.12/library/concurrent.futures.html

[9] Preferred Networks, "CuPy documentation." https://docs.cupy.dev/

[10] J. Sanders, "Veusz — a scientific plotting package." https://veusz.github.io/

[11] M. Ottosson *et al.*, "Qt.py," v2.0.7. https://github.com/mottosso/Qt.py

[12] R. Regan, "Number of digits required for round-trip conversions," Exploring Binary. https://www.exploringbinary.com/number-of-digits-required-for-round-trip-conversions/

[13] W. W. Wallace, "vszh5_writer.py — Veusz HDF5 document writer," v0.0.20, NRAO/GBO, 2026.